

本小节内容

课时 14 作业 1

课时 14 作业 2

课时 14 作业 1

Description

读取字符串 abcdefghij，然后层次建树建立一颗二叉树，然后前序遍历输出 abdhiejcfg，注意不要打印前序遍历几个汉字

Input

abcdefghijkl

Output

abdhiejcfg

这道题考察的是二叉树建树，前序遍历，和上课讲解的代码是类似的，下面直接上代码

答案如下：

```
#include <stdio.h>
#include <stdlib.h>
typedef char BiElemType;
typedef struct BiTNode {
    BiElemType c;//c 就是书籍上的 data
    struct BiTNode* lchild;
    struct BiTNode* rchild;
}BiTNode, *BiTree;

typedef struct tag {
    BiTree p;//树的某一个结点的地址值
    struct tag* pnext;
```

```
}tag_t, *ptag_t;

void preOrder(BiTree p)
{
    if (p != NULL)
    {
        putchar(p->c); //等价于 visit 函数
        preOrder(p->lchild);
        preOrder(p->rchild);
    }
}

int main()
{
    BiTree pnew;
    int i, j, pos;
    char c;
    BiTree tree = NULL; //树根
    ptag_t phead = NULL, ptail = NULL, listpnew = NULL, pcur = NULL; //phead 就是队列头, ptail 就是队
    列尾
    //abcdefghij
    while (scanf("%c", &c) != EOF)
    {
        if (c == '\n')
        {
            break;
        }
        pnew = (BiTree)calloc(1, sizeof(BiTNode)); //calloc 申请空间并对空间进行初始化, 赋值为 0
        pnew->c = c; //数据放进去
        listpnew = (ptag_t)calloc(1, sizeof(tag_t)); //给队列结点申请空间
        listpnew->p = pnew;
        if (NULL == tree)
        {
            tree = pnew; //树的根
            phead = listpnew; //队列头
            ptail = listpnew; //队列尾
            pcur = listpnew;
            continue;
        }
        else {
            ptail->pnext = listpnew; //新结点放入链表, 通过尾插法
            ptail = listpnew; //ptail 指向队列尾部
        } //pcur 始终指向要插入的结点的位置
        if (NULL == pcur->p->lchild) //如何把新结点放入树
        {
```

```
        pcur->p->lchild = pnew;//把新结点放到要插入结点的左边
    }
    else if (NULL == pcur->p->rchild)
    {
        pcur->p->rchild = pnew;//把新结点放到要插入结点的右边
        pcur = pcur->pnext;//左右都放了结点后，pcur 指向队列的下一个
    }
}
preOrder(tree);
return 0;
}
```

课时 14 作业 2

Description

读取字符串 abcdefghij，然后层次建树建立一颗二叉树，然后中序遍历输出 hdibjeafc g，后序遍历输出 hidjebfgca，层序遍历输出 abcdefghij，注意不要输出汉字

Input

abcdefghijkl

Output

中序遍历输出 hdibjeafc g，后序遍历输出 hidjebfgca，层序遍历输出 abcdefghij，每个一行

这道题的解法与上课写的代码一致，但是由于 OJ 只能提交一个代码文件，所以我们全部写入 main.cpp 内

```
#include <stdio.h>
#include <stdlib.h>
typedef char DataType;
typedef struct TreeNode {
    DataType data;
    struct TreeNode* lchild;
    struct TreeNode* rchild;
}TreeNode, *Tree;

typedef Tree LinkDataType;
```

//存储树的结点地址

```
typedef struct tag {  
    Tree p;//树的某一个结点地址值  
    struct tag* pnext;  
}tag_t, *ptag_t;  
//队列的相关数据结构
```

```
typedef struct LinkNode {  
    LinkDataType data;  
    struct LinkNode* next;  
}LinkNode;  
typedef struct {  
    LinkNode* front, * rear;  
}LinkQueue;
```

//初始化队列

```
void InitQueue(LinkQueue& Q) {  
    Q.front = Q.rear = (LinkNode*)malloc(sizeof(LinkNode));  
    Q.front->next = NULL;  
}
```

//判断队列是否为空

```
bool IsEmpty(LinkQueue Q) {  
    if (Q.front == Q.rear) {  
        return true;  
    }  
    else  
    {  
        return false;  
    }  
}
```

//进队

```
void EnQueue(LinkQueue& Q, LinkDataType x) {  
    LinkNode* s = (LinkNode*)malloc(sizeof(LinkNode));  
    s->data = x; s->next = NULL;  
    Q.rear->next = s;  
    Q.rear = s;  
}
```

//出队

```
bool DeQueue(LinkQueue& Q, LinkDataType& x) {  
    if (Q.front == Q.rear) {  
        return false;  
    }  
    else  
    {
```

LinkNode* p = Q.front->next; //头结点什么都没有存，所以头结点的下一个节点才有数据

```
x = p->data;
Q.front->next = p->next;
if (Q.rear == p) {
    Q.rear = Q.front;
    free(p);
    return true;
}
}
return true; //这里要注意，不加会 wrong answer
}
//层序建树 (放入字符的树)
void BuildCharTree(Tree& T)
{
    Tree pnew;
    char x;
    ptag_t phead = NULL, ptail = NULL, pcur = NULL, listpnew = NULL; //phead 队列头 ptail 队列尾
    while (scanf("%c", &x) != EOF)
    {
        if (x == '\n') {
            break;
        }
        //calloc 申请空间并对空间进行初始化，赋值为 0
        //初始化结点
        pnew = (Tree)calloc(1, sizeof(TreeNode));
        pnew->data = x;
        //给队列结点申请空间
        listpnew = (ptag_t)calloc(1, sizeof(ptag_t));
        listpnew->p = pnew;

        if (NULL == T) {
            T = pnew; //树的根
            phead = listpnew; //队列头
            ptail = listpnew; //队列尾
            pcur = listpnew;
            continue;
        }
        else //T!=NULL
        {
            //将新结点放入链表，尾插法
            ptail->pnext = listpnew;
            ptail = listpnew;
        }
        //pcur 始终指向要插入的结点位置
        if (NULL == pcur->p->lchild) {
```

```
        //把新结点放到要插入结点的左边
        pcur->p->lchild = pnew;
    }
    else if (NULL == pcur->p->rchild) {
        //把新结点放到要插入结点的右边
        pcur->p->rchild = pnew;
        //左右都放了结点后，pcur 指向队列的下一个
        pcur = pcur->pnext;
    }
}
}
```

//中序遍历

```
void InOrder(Tree T)
{
    if (NULL != T) {
        InOrder(T->lchild);
        putchar(T->data);
        InOrder(T->rchild);
    }
}
```

//后序遍历

```
void BackOrder(Tree T)
{
    if (NULL != T) {
        BackOrder(T->lchild);
        BackOrder(T->rchild);
        putchar(T->data);
    }
}
```

//层次遍历,层序遍历，广度优先遍历

```
void LevelOrder(Tree T)
{
    LinkQueue Q;
    InitQueue(Q);
    Tree p;
    EnQueue(Q, T); //树根入队
    while (!IsEmpty(Q))
    {
        DeQueue(Q, p);
```

```
        putchar(p->data);
        if (p->lchild != NULL)
            EnQueue(Q, p->lchild);
        if (p->rchild != NULL)
            EnQueue(Q, p->rchild);
    }
}
int main()
{
    Tree T = NULL;
    //层次建树  abcdefghij
    BuildCharTree(T);

    //中序遍历
    InOrder(T);
    printf("\n");
    //后序遍历
    BackOrder(T);
    printf("\n");
    //层序遍历
    LevelOrder(T);
    return 0;
}
```